

Security Experiment Worksheet

Grand Marina Hotel — TLS Security Assessment

Name: Sok Leng Chan Date: 07/26/2026

Experiment 1: Eavesdropper Test

Question: Can someone spy on our sensor data?

Scenario	What You Observed
Without TLS (Part A)	<i>Example: "Saw all 5 messages with device IDs and pressure readings in plain text"</i>
With TLS (Part B)	<i>Example: "Error: A TLS error occurred. Could not see any messages."</i>

Conclusion (Did TLS block eavesdropping?):

Yes, TLS blocks eavesdropping. As you saw below screenshot, port error for eavesdrop command on the second terminal.

Screenshot:

```
Windows PowerShell
PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project> mosquitto_sub -h localhost -p 8883 -t "grandmarina/#" -v
Error: protocol error
PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project>

Windows PowerShell
PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project> python experiment_runner.py --mode publish --tls on --count 5

=====
Publishing 5 messages
TLS: ON
Port: 8883
=====

Published message 1/5: pressure=82.5 PSI
Published message 2/5: pressure=83.5 PSI
Published message 3/5: pressure=84.5 PSI
Published message 4/5: pressure=85.5 PSI
Published message 5/5: pressure=86.5 PSI

Done publishing!
PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project>
```

Experiment 2: Certificate Test

Question: What happens when certificates are wrong?

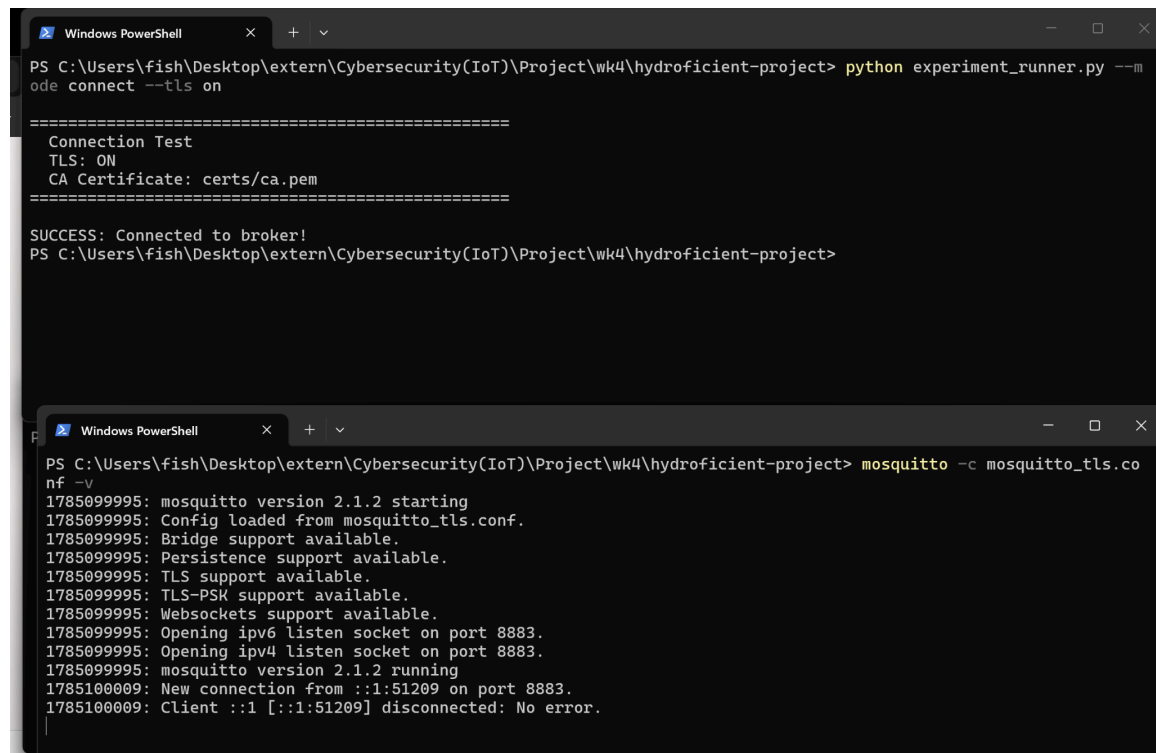
Scenario	Result	Error Message (if any)
A: Correct certificates	SUCCESS	
B: Wrong CA	FAILED	SSL: certificate failed
C: No verification	SUCCESS	

Conclusion (What does this tell you about certificate validation?):

Certificate validation test reveals if the wrong CA tried to get access, it was denied, but not just the correct CA can get access. Any CA which was certificated can get access, it was dangerous, so we need to be aware of that.

Screenshot:

Scenario A: Correct Certificates



```
Windows PowerShell
PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project> python experiment_runner.py --mode connect --tls on

=====
Connection Test
TLS: ON
CA Certificate: certs/ca.pem
=====

SUCCESS: Connected to broker!
PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project>

Windows PowerShell
PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project> mosquitto -c mosquitto_tls.conf -v
1785099995: mosquitto version 2.1.2 starting
1785099995: Config loaded from mosquitto_tls.conf.
1785099995: Bridge support available.
1785099995: Persistence support available.
1785099995: TLS support available.
1785099995: TLS-PSK support available.
1785099995: Websockets support available.
1785099995: Opening ipv6 listen socket on port 8883.
1785099995: Opening ipv4 listen socket on port 8883.
1785099995: mosquitto version 2.1.2 running
1785100009: New connection from ::1:51209 on port 8883.
1785100009: Client ::1 [::1:51209] disconnected: No error.
```

Scenario B: Wrong CA Certificate

```
Windows PowerShell
PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project> python experiment_runner.py --mode generate-wrong-ca

Generating wrong CA certificate for testing...
Saved: certs/wrong-ca.pem
To test: Use this as ca_certs in your client
PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project> python experiment_runner.py --mode test-wrong-ca
Testing connection with wrong CA...

=====
Connection Test
TLS: ON
CA Certificate: certs/wrong-ca.pem
=====

FAILED: [SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed: self-signed certificate in certificate chain (_ssl.c:1081)
PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project>

Windows PowerShell
PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project> mosquitto -c mosquitto_tls.conf -v
1785099995: mosquitto version 2.1.2 starting
1785099995: Config loaded from mosquitto_tls.conf.
1785099995: Bridge support available.
1785099995: Persistence support available.
1785099995: TLS support available.
1785099995: TLS-PSK support available.
1785099995: Websockets support available.
1785099995: Opening ipv6 listen socket on port 8883.
1785099995: Opening ipv4 listen socket on port 8883.
1785099995: mosquitto version 2.1.2 running
1785100009: New connection from ::1:51209 on port 8883.
1785100009: Client ::1 [::1:51209] disconnected: No error.
1785100133: New connection from ::1:51844 on port 8883.
1785100133: OpenSSL Error while trying to get the error[0]: error:0A000418:SSL routines::tlsv1 alert unknown ca
1785100133: Client ::1 [::1:51844] disconnected: protocol error.
```

Scenario C: No Certificate Verification

```
Windows PowerShell
PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project> python experiment_runner.py --mode connect --tls on --no-ca

=====
Connection Test
TLS: ON
CA Certificate: NONE
=====

SUCCESS: Connected to broker!
PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project>

Windows PowerShell
PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project> mosquitto -c mosquitto_tls.conf -v
1785099995: mosquitto version 2.1.2 starting
1785099995: Config loaded from mosquitto_tls.conf.
1785099995: Bridge support available.
1785099995: Persistence support available.
1785099995: TLS support available.
1785099995: TLS-PSK support available.
1785099995: Websockets support available.
1785099995: Opening ipv6 listen socket on port 8883.
1785099995: Opening ipv4 listen socket on port 8883.
1785099995: mosquitto version 2.1.2 running
1785100009: New connection from ::1:51209 on port 8883.
1785100009: Client ::1 [::1:51209] disconnected: No error.
1785100133: New connection from ::1:51844 on port 8883.
1785100133: OpenSSL Error while trying to get the error[0]: error:0A000418:SSL routines::tlsv1 alert unknown ca
1785100133: Client ::1 [::1:51844] disconnected: protocol error.
1785100210: New connection from ::1:53565 on port 8883.
1785100210: Client ::1 [::1:53565] disconnected: No error.
```

Experiment 3: Speed Test

Question: Does encryption slow things down?

Metric	TLS OFF	TLS ON
Average latency	50.88 ms	50.98 ms
Min latency	0.62 ms	0.74 ms
Max latency	101.17 ms	101.21 ms

Calculated TLS overhead: 0.197% $(50.98 - 50.88) / 50.88 \times 100\% = 0.197\%$

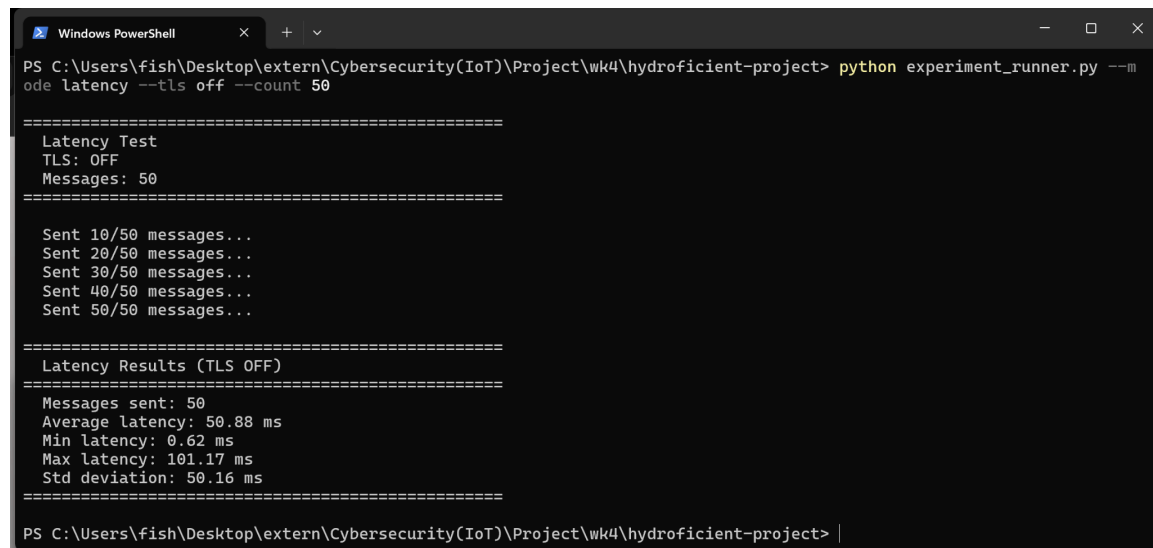
Formula: TLS Overhead = (TLS_avg - NoTLS_avg) / NoTLS_avg x 100%

Conclusion (Is this overhead acceptable for Grand Marina's sensors?):

Yes, TLS overhead is 0.197%. It is less than 50%, so it is totally acceptable.

Screenshot:

Part A: Without TLS



```
Windows PowerShell
PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project> python experiment_runner.py --mode latency --tls off --count 50

=====
Latency Test
TLS: OFF
Messages: 50
=====

Sent 10/50 messages...
Sent 20/50 messages...
Sent 30/50 messages...
Sent 40/50 messages...
Sent 50/50 messages...

=====
Latency Results (TLS OFF)
=====
Messages sent: 50
Average latency: 50.88 ms
Min latency: 0.62 ms
Max latency: 101.17 ms
Std deviation: 50.16 ms
=====

PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project> |
```

Part B: With TLS

```
Windows PowerShell
PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project> python experiment_runner.py --mode latency --tls on --count 50

=====
Latency Test
TLS: ON
Messages: 50
=====

Sent 10/50 messages...
Sent 20/50 messages...
Sent 30/50 messages...
Sent 40/50 messages...
Sent 50/50 messages...

=====
Latency Results (TLS ON)
=====
Messages sent: 50
Average latency: 50.98 ms
Min latency: 0.74 ms
Max latency: 101.21 ms
Std deviation: 50.09 ms
=====

PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project>
```

Experiment 4: Stress Test

Question: Can the system handle Grand Marina's load?

Rate	TLS OFF	TLS ON
10 msg/sec (Normal)	SUCCESS	SUCCESS
25 msg/sec (Moderate)	SUCCESS	SUCCESS
50 msg/sec (Emergency)	SUCCESS	SUCCESS
100 msg/sec (Beyond req.)	SUCCESS	SUCCESS

Conclusion (Can the TLS-secured system handle Grand Marina's load requirements?):

Yes, As I saw below screenshots from the stress test. Every situation is a success to achieve the goals. They don't have any delay even though TLS is on. So I think the system can handle the larger volume data which Grand Marina passes through.

Screenshot:

10 msg/sec (Normal operation) without TLS

```
Windows PowerShell
20s: sent=208, rate=10.0/sec
20s: sent=209, rate=10.0/sec
25s: sent=250, rate=10.0/sec
25s: sent=251, rate=10.0/sec
25s: sent=252, rate=10.0/sec
25s: sent=253, rate=10.0/sec
25s: sent=254, rate=10.0/sec
25s: sent=255, rate=10.0/sec
25s: sent=256, rate=10.0/sec
25s: sent=257, rate=10.0/sec
25s: sent=258, rate=10.0/sec
25s: sent=259, rate=10.0/sec
30s: sent=300, rate=10.0/sec

=====
Stress Test Results (TLS OFF)
=====
Target rate: 10 msg/sec
Actual rate: 9.7 msg/sec
Messages sent: 300
Errors: 0
Success rate: 100.0%
Status: SUCCESS (achieved target rate)
=====

PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project>
```

10 msg/sec (Normal operation) with TLS

```
Windows PowerShell
20s: sent=208, rate=10.0/sec
20s: sent=209, rate=10.0/sec
25s: sent=250, rate=10.0/sec
25s: sent=251, rate=10.0/sec
25s: sent=252, rate=10.0/sec
25s: sent=253, rate=10.0/sec
25s: sent=254, rate=10.0/sec
25s: sent=255, rate=10.0/sec
25s: sent=256, rate=10.0/sec
25s: sent=257, rate=10.0/sec
25s: sent=258, rate=10.0/sec
25s: sent=259, rate=10.0/sec
30s: sent=300, rate=10.0/sec

=====
Stress Test Results (TLS ON)
=====
Target rate: 10 msg/sec
Actual rate: 9.7 msg/sec
Messages sent: 300
Errors: 0
Success rate: 100.0%
Status: SUCCESS (achieved target rate)
=====

PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project> |
```

25 msg/sec (Moderate load) without TLS

```
Windows PowerShell
25s: sent=638, rate=25.0/sec
25s: sent=639, rate=25.0/sec
25s: sent=640, rate=25.0/sec
25s: sent=641, rate=25.0/sec
25s: sent=642, rate=25.0/sec
25s: sent=643, rate=25.0/sec
25s: sent=644, rate=25.0/sec
25s: sent=645, rate=25.0/sec
25s: sent=646, rate=25.0/sec
25s: sent=647, rate=25.0/sec
25s: sent=648, rate=25.0/sec
25s: sent=649, rate=25.0/sec
30s: sent=750, rate=25.0/sec

=====
Stress Test Results (TLS OFF)
=====
Target rate: 25 msg/sec
Actual rate: 24.2 msg/sec
Messages sent: 750
Errors: 0
Success rate: 100.0%
Status: SUCCESS (achieved target rate)
=====

PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project> |
```

25 msg/sec (Moderate load) with TLS

```
Windows PowerShell
25s: sent=638, rate=25.0/sec
25s: sent=639, rate=25.0/sec
25s: sent=640, rate=25.0/sec
25s: sent=641, rate=25.0/sec
25s: sent=642, rate=25.0/sec
25s: sent=643, rate=25.0/sec
25s: sent=644, rate=25.0/sec
25s: sent=645, rate=25.0/sec
25s: sent=646, rate=25.0/sec
25s: sent=647, rate=25.0/sec
25s: sent=648, rate=25.0/sec
25s: sent=649, rate=25.0/sec
30s: sent=750, rate=25.0/sec

=====
Stress Test Results (TLS ON)
=====
Target rate: 25 msg/sec
Actual rate: 24.2 msg/sec
Messages sent: 750
Errors: 0
Success rate: 100.0%
Status: SUCCESS (achieved target rate)
=====

PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project> |
```

100 msg/sec (Beyond requirements) without TLS

```
Windows PowerShell
25s: sent=2588, rate=100.0/sec
25s: sent=2589, rate=100.0/sec
25s: sent=2590, rate=100.0/sec
25s: sent=2591, rate=100.0/sec
25s: sent=2592, rate=100.0/sec
25s: sent=2593, rate=100.0/sec
25s: sent=2594, rate=100.0/sec
25s: sent=2595, rate=100.0/sec
25s: sent=2596, rate=100.0/sec
25s: sent=2597, rate=100.0/sec
25s: sent=2598, rate=100.0/sec
25s: sent=2599, rate=100.0/sec
30s: sent=3000, rate=100.0/sec

=====
Stress Test Results (TLS OFF)
=====
Target rate: 100 msg/sec
Actual rate: 96.8 msg/sec
Messages sent: 3000
Errors: 0
Success rate: 100.0%
Status: SUCCESS (achieved target rate)
=====

PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project> |
```

100 msg/sec (Beyond requirements) with TLS

```
Windows PowerShell
25s: sent=2588, rate=100.0/sec
25s: sent=2589, rate=100.0/sec
25s: sent=2590, rate=100.0/sec
25s: sent=2591, rate=100.0/sec
25s: sent=2592, rate=100.0/sec
25s: sent=2593, rate=100.0/sec
25s: sent=2594, rate=100.0/sec
25s: sent=2595, rate=100.0/sec
25s: sent=2596, rate=100.0/sec
25s: sent=2597, rate=100.0/sec
25s: sent=2598, rate=100.0/sec
25s: sent=2599, rate=100.0/sec
30s: sent=3000, rate=100.0/sec

=====
Stress Test Results (TLS ON)
=====
Target rate: 100 msg/sec
Actual rate: 96.8 msg/sec
Messages sent: 3000
Errors: 0
Success rate: 100.0%
Status: SUCCESS (achieved target rate)
=====

PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project>
```


50 msg/sec (Emergency mode) without TLS

```
Windows PowerShell
25s: sent=1288, rate=50.0/sec
25s: sent=1289, rate=50.0/sec
25s: sent=1290, rate=50.0/sec
25s: sent=1291, rate=50.0/sec
25s: sent=1292, rate=50.0/sec
25s: sent=1293, rate=50.0/sec
25s: sent=1294, rate=50.0/sec
25s: sent=1295, rate=50.0/sec
25s: sent=1296, rate=50.0/sec
25s: sent=1297, rate=50.0/sec
25s: sent=1298, rate=50.0/sec
25s: sent=1299, rate=50.0/sec
30s: sent=1500, rate=50.0/sec

=====
Stress Test Results (TLS OFF)
=====
Target rate: 50 msg/sec
Actual rate: 48.4 msg/sec
Messages sent: 1500
Errors: 0
Success rate: 100.0%
Status: SUCCESS (achieved target rate)
=====

PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project>
```

50 msg/sec (Emergency mode) with TLS

```
Windows PowerShell
25s: sent=1288, rate=50.0/sec
25s: sent=1289, rate=50.0/sec
25s: sent=1290, rate=50.0/sec
25s: sent=1291, rate=50.0/sec
25s: sent=1292, rate=50.0/sec
25s: sent=1293, rate=50.0/sec
25s: sent=1294, rate=50.0/sec
25s: sent=1295, rate=50.0/sec
25s: sent=1296, rate=50.0/sec
25s: sent=1297, rate=50.0/sec
25s: sent=1298, rate=50.0/sec
25s: sent=1299, rate=50.0/sec
30s: sent=1500, rate=50.0/sec

=====
Stress Test Results (TLS ON)
=====
Target rate: 50 msg/sec
Actual rate: 48.4 msg/sec
Messages sent: 1500
Errors: 0
Success rate: 100.0%
Status: SUCCESS (achieved target rate)
=====

PS C:\Users\fish\Desktop\extern\Cybersecurity(IoT)\Project\wk4\hydroficient-project>
```